

ESTRUCTURA DE CÓDIGOS DEL SISTEMA

Documento técnico para exposición y defensa del **Sistema Inteligente de Producción Láctea**.
Elaborado a partir del código activo y contrastado con `GUIA_SISTEMA.md`.
Fecha de revisión: 24 de septiembre de 2026.

Índice

- 1. [Cómo leer este documento](#)
- 2. [Arquitectura y recorrido general](#)
- 3. [Mapa general de módulos](#)
- 4. [Producción](#)
- 5. [POS y carrito de ventas](#)
- 6. [Inventario por lote e inventario general](#)
- 7. [Ajustes de inventario](#)
- 8. [Descartes de productos](#)
- 9. [Proveedores y entradas de inventario](#)
- 10. [Consignación histórica](#)
- 11. [Productos y presentaciones](#)
- 12. [Sensores IoT y API del ESP32](#)
- 13. [Control de actuadores](#)
- 14. [Dashboard, alertas y eventos](#)
- 15. [Reportes](#)
- 16. [Usuarios, autenticación y roles](#)
- 17. [Catálogo público](#)
- 18. [Transacciones, COMMIT, ROLLBACK y concurrencia](#)
- 19. [Mapa completo de tablas SQL](#)
- 20. [Relaciones principales](#)
- 21. [Rutas y archivos transversales](#)
- 22. [Resumen para la defensa](#)

1. Cómo leer este documento

1.1 Raíz absoluta del proyecto

Todas las rutas indicadas en este documento parten de:

```
D:\xampp\htdocs\web3\ProyectoFuncional_7_17_2026_version12\9_11_2026_monitoreo_lacteo_sistema_ventas_funcional_v2
```

Cuando una celda muestra `app/...`, `resources/...`, `routes/...` o `database/...`, la ruta completa se obtiene anexándola a la raíz anterior. En la primera fila de cada módulo se incluye además la ruta absoluta de su pieza principal.

1.2 Significado de las operaciones

Marca	Significado real
SELECT	Lectura mediante Eloquent o Query Builder: <code>where</code> , <code>with</code> , <code>get</code> , <code>first</code> , <code>paginate</code> , <code>count</code> , <code>exists</code> , <code>find</code> o agregaciones.
INSERT	Creación mediante <code>Model::create()</code> , una relación <code>->create()</code> , <code>insert()</code> o <code>insertGetId()</code> .
UPDATE	Cambio mediante <code>update()</code> , <code>save()</code> , <code>increment()</code> o <code>decrement()</code> .
DELETE	Eliminación física mediante <code>delete()</code> .
BAJA LÓGICA	No elimina la fila: ejecuta un <code>UPDATE activo = false</code> o usa <code>SoftDeletes</code> .
SIN SQL	Renderizado, cálculo, caché, redirección o validación que no escribe una tabla directamente.
TRANSACCIÓN	Bloque <code>DB::transaction()</code> : Laravel confirma con COMMIT si termina bien y revierte con ROLLBACK si se lanza una excepción.

1.3 Capas usadas por el sistema

Capa	Ubicación	Responsabilidad
Interfaz	<code>resources/views/</code> y <code>resources/js/</code>	Botones, formularios, tablas, carrito, peticiones <code>fetch</code> y visualización.
Rutas	<code>routes/web.php</code> , <code>routes/api.php</code>	Relacionan método HTTP y URL con un controlador y aplican middleware.
Controlador	<code>app/Http/Controllers/</code>	Valida la petición, coordina servicios/modelos y decide la respuesta.
Servicio	<code>app/Services/</code>	Contiene reglas complejas, transacciones, FEFO, bloqueos, cálculos y consultas reutilizables.
Modelo	<code>app/Models/</code>	Representa una tabla y sus relaciones Eloquent.
Base de datos	<code>database/migrations/</code>	Define tablas, columnas, claves foráneas e índices.

2. Arquitectura y recorrido general

2.1 Flujo web normal

Clic o envío de formulario

→ Blade/JavaScript

→ ruta de routes/web.php

→ middleware auth/rol/OperadorComercial

→ método del Controller

→ validación Request

→ Service o Model Eloquent

→ SELECT / INSERT / UPDATE / DELETE en MySQL

→ vista, redirección, JSON, PDF, CSV o Excel

2.2 Flujo IoT

ESP32

→ HTTP/JSON /api/*

→ routes/api.php + throttle:60,1

→ Api\Esp32Controller

→ Sensor / Dispositivo / Produccion / Alerta / Actuador

→ MySQL y caché

→ respuesta JSON al dispositivo

2.3 Flujo de una operación transaccional

Controller valida

→ Service abre DB::transaction

→ SELECT ... FOR UPDATE mediante lockForUpdate()

→ verifica reglas y saldo

→ INSERT/UPDATE/DELETE relacionados

→ éxito: COMMIT automático

→ excepción: ROLLBACK automático de todo el bloque

3. Mapa general de módulos

Módulo	URL principal	Controlador principal	Modelos/tablas centrales	Vista principal
Producción	GET /produccion	ProduccionController	Produccion → producciones; Producto, Dispositivo, Evento, Actuador	resources/views/produccion/index.blade.php
POS / carrito	GET /pos	ComercialController + ComercialService	Presentacion, IngresoProductorItem, TicketVenta, Venta	resources/views/pos/index.blade.php
Inventario	GET /inventario, /inventario-general	ComercialController + InventarioConsulta	ingresos_productores_items, presentaciones, movimientos_inventario	resources/views/inventario/*.blade.php
Ajustes	/ajustes-inventario	AjusteInventarioController + AjusteInventarioService	ajustes_inventario, lotes, presentaciones, movimientos	resources/views/ajustes-inventario/
Descartes	/descartes-productos	DescarteProductoController + DescarteProductoService	descartes_productos, lotes, presentaciones, movimientos	resources/views/descartes-productos/
Proveedores	/productores	ProductorController	productores, ingresos_productores	resources/views/productores/
Entradas/acopio	/ingresos-productores	ComercialController + ComercialService	ingresos_productores, ingresos_productores_items	resources/views/ingresos/
Consignación	Redirección a /ingresos-productores	No existe controlador vigente	Tablas históricas eliminadas/renombradas	No existe vista vigente
Productos	/productos	ProductoController	productos	resources/views/productos/
Presentaciones	/presentaciones	PresentacionController	presentaciones	resources/views/presentaciones/
IoT/ESP32	/api/*	Api\Esp32Controller	dispositivos, sensores, actuadores, producciones, alertas	JSON; dashboard/control como GUI
Control	/control	ControlController + ActuadorService	actuadores, sensores, eventos	resources/views/control/index.blade.php
Dashboard	/dashboard	DashboardController, Api\Esp32Controller	múltiples tablas IoT	resources/views/dashboard/index.blade.php
Alertas/eventos	/alertas, /eventos	AlertaController, EventoController	alertas, eventos	resources/views/alertas/, eventos/
Reportes	/reportes*	ReporteController, ReporteComercialController	producción, ventas, entradas, movimientos	resources/views/reportes/
Usuarios y acceso	/login, /usuarios	AuthController, UserController	users, sessions	resources/views/auth/, usuarios/
Catálogo público	/catalogo	CatalogoController	presentaciones, productos, productores	resources/views/catalogo/

4. Producción

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Abrir pantalla de producción	Ruta: routes/web.php → GET /produccion. Controller: app/Http/Controllers/ProduccionController.php. Modelos: User, Producto, Produccion. Vista: resources/views/produccion/index.blade.php. JS: resources/js/produccion.js.	ProduccionController::index()	SELECT en users, productos y producciones.	Usuario entra desde el menú → la ruta autenticada ejecuta index() → carga operadores/productos activos y producción en proceso → renderiza formulario y estado.
Iniciar producción	POST /produccion/iniciar → mismo controller/modelos/vista. Servicios: EventoService, ActuadorService.	ProduccionController::iniciar(); auxiliares validarInicio(), calcularInsumoRequerido(), detalleInsumo().	TRANSACCIÓN: SELECT productos y dispositivos; INSERT producciones. Después del bloque: INSERT eventos; posibles UPDATE actuadores. SELECT y UPDATE en producciones; INSERT en eventos; UPDATE en actuadores. No está dentro de una única transacción.	Clic “Iniciar” → formulario valida operador, producto, leche y temperatura → comprueba que no exista otra producción activa y que exista ESP32 → calcula cuajo → crea producción → registra evento → enciende motor/ventilador si están en automático.
Finalizar producción manualmente	POST /produccion/finalizar → controller; vista produccion/index.blade.php.	ProduccionController::finalizar()	UPDATE en producciones; INSERT en eventos; UPDATE en actuadores. No está dentro de una única transacción.	Clic “Finalizar” → busca la producción activa → cambia estado y fecha fin → limpia caché estadística → registra evento → apaga actuadores → vuelve a la pantalla.
Recomendación de cuajo en navegador	resources/js/produccion.js y resources/js/rennet-recommendation.js.	initializeProductionForm(), calcularInsumos() interno, requiredRennet(), rennetReferences().	SIN SQL; cálculo visual. El servidor recalcula el valor definitivo.	Al elegir producto o cambiar litros, JS calcula una recomendación; al enviar, iniciar() vuelve a calcular para no confiar en el navegador.
Finalización automática por temperatura	POST /api/finalizar-produccion o lógica dentro de POST /api/temperatura. Controller: app/Http/Controllers/Api/Esp32Controller.php.	finalizarProduccionAutomatica() y temperatura()	UPDATE producciones; UPDATE actuadores; INSERT eventos y alertas.	ESP32 envía temperatura → controlador valida → si se cumple condición térmica, finaliza producción, apaga actuadores, registra auditoría y responde JSON.

Defensa: el inicio sí usa DB::transaction(). El evento y el encendido automático se ejecutan después del COMMIT de la producción, por lo que no forman parte de la misma transacción SQL.

5. POS y carrito de ventas

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Abrir POS	GET /pos en routes/web.php. Controller: ComercialController. Vista: resources/views/pos/index.blade.php. JS: resources/js/comercial.js.	ComercialController::pos()	SIN SQL; devuelve la vista.	Usuario abre Ventas → Blade presenta catálogo y carrito vacíos → JS inicia la carga del catálogo.
Cargar catálogo vendible	GET /comercial/catalogo. Controller: ComercialController. Modelos: Presentacion, Producto, IngresoProductorItem, IngresoProductor, Productor.	ComercialController::catalogo(); JS initializeComercial() → load() → api().	SELECT en presentaciones, productos, ingresos_productores_items, ingresos_productores, productores; solo activos, stock positivo, ingreso abierto y caducidad vigente.	JS hace fetch('/comercial/catalogo') → servidor carga presentaciones y lotes → agrupa por proveedor → devuelve JSON → render() dibuja tarjetas.
Agregar/quitar del carrito	Solo navegador: resources/js/comercial.js; apoyo resources/js/supplier-cart.js.	setQty(), render(), allocations(), subtotal(); helpers chooseSupplier(), supplierStock(), supplierPrice().	SIN SQL; usa Map cart en memoria del navegador.	Clic “Agregar”, +, - o cambio de cantidad → JS modifica cart → calcula estimación FEFO y subtotal → todavía no guarda nada.
Cobrar carrito	POST /comercial/ventas. Controller: ComercialController::vender(). Service: ComercialService::vender(). Modelos: Venta, Presentacion, IngresoProductorItem. Vista/JS: pos/index.blade.php, comercial.js::pay().	ComercialController::vender() → ComercialService::vender(); auxiliares presentaciones(), stock(), ticket(), repetir().	TRANSACCIÓN: SELECT FOR UPDATE presentaciones y ingresos_productores_items; SELECT / INSERT / UPDATE tickets_venta; INSERT ventas; UPDATE lotes y presentaciones.stock.	Clic Efectivo/QR/Crédito → JS genera/envía UUID, pago, cliente e ítems → valida → abre transacción → bloquea presentaciones → controla idempotencia → selecciona lotes FEFO → crea líneas → descuenta lote y stock global → actualiza total → COMMIT → devuelve ticket JSON.
FEFO	Service app/Services/ComercialService.php.	Dentro de ComercialService::vender()	SELECT ... ORDER BY fecha_caducidad, id FOR UPDATE sobre ingresos_productores_items.	El servidor consume primero el lote que vence antes. La estimación del navegador no manda: la decisión definitiva se repite bajo bloqueo en MySQL.

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Idempotencia	Misma ruta y servicio.	<code>vender() , repetir() , ticket()</code>	<code>SELECT por tickets_venta.clave ; INSERT con UUID único; comparación payload_hash .</code>	Si se reintenta exactamente la misma venta, retorna el ticket anterior; si la misma clave contiene otros datos, rechaza la operación.
Ticket y vista imprimible	JSON de <code>ComercialService::ticket() ; JS buildPrintableTicket() .</code>	<code>ticket() , buildPrintableTicket()</code>	<code>SELECT tickets_venta , ventas , lotes , presentaciones , productos y productores.</code>	Tras COMMIT, el servidor devuelve líneas reales por lote → JS agrupa por producto/precio → abre modal y permite imprimir.

Regla de rollback del carrito

Si un artículo no tiene existencia suficiente, `ComercialService::error()` lanza `ValidationException`. Al salir con excepción de `DB::transaction()`, Laravel revierte el ticket, todas las líneas de venta y todos los descuentos ya hechos dentro de ese carrito. Por eso el mensaje indica: “Se revirtió todo el carrito.”

6. Inventario por lote e inventario general

Archivo principal absoluto:

D:\xampp\htdocs\web3\ProyectoFuncional_7_17_2026_version12\9_11_2026_monitoreo_lacteo_sistema_ventas_funcional_v2\app\Services\InventarioConsulta.php

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Inventario por lote	GET /inventario . Controller: ComercialController . Service: InventarioConsulta . Vista: resources/views/inventario/index.blade.php .	ComercialController::inventario() → InventarioConsulta::lotes() y aplicarFiltrosLotes() .	SELECT en ingresos_productores_items , presentaciones , productos , ingresos_productores , productores ; SELECT de últimos movimientos_inventario .	Usuario filtra producto, proveedor o fechas → Reques normaliza filtros → servicio construye consulta → pagina lotes y muestra kardex reciente.
Inventario consolidado	GET /inventario-general . Vista: resources/views/inventario/general.blade.php .	ComercialController::inventarioGeneral() → InventarioConsulta::grupos() , totales() , detalleGrupo() , consultaGeneralAgrupada() .	SELECT con JOIN , SUM(CASE...) , COUNT y GROUP BY sobre lotes , entradas , productores , presentaciones y productos.	Usuario abre el resumen → se agrupa por presentación y proveedor → calcula vendible, próximo a vencer, no vendible y físico → opcionalmente abre detalle FEFO.
Movimiento manual por lote	POST /lotes/{item}/movimientos . en inventario/index.blade.php . Controller/Service: ComercialController , ComercialService .	ComercialController::movimiento() → ComercialService::movimiento() .	TRANSACCIÓN: SELECT FOR UPDATE en presentaciones , ingresos_productores , lotes ; UPDATE lote y presentación ; INSERT movimientos_inventario .	Usuario abre “Movimiento”, elige entrada/salida/ajuste y motivo → valida rol/cantidad → bloquea registros → evita saldo negativo o lote vencido → actualiza ambos saldos y escribe auditoría → COMMIT.
Regularizar datos faltantes	PATCH /lotes/{item}/regularizar . Formulario en resources/views/ingresos/show.blade.php .	ComercialController::regularizar() → ComercialService::regularizar() .	TRANSACCIÓN: SELECT FOR UPDATE ; UPDATE ingresos_productores_items ; INSERT movimientos_inventario .	Administrador completa fecha o costo previamente nulo → el servicio impide reemplazar un dato conocido → guarda dato faltante y deja motivo en kardex.
Búsqueda flexible de presentación	app/Services/BusquedaPresentacion.php .	BusquedaPresentacion::aplicar()	SELECT filtrado sobre presentaciones y productos ; usa LIKE , igualdad de unidad/contenido.	El texto se separa en términos ; reconoce unidades como L, ml, g o kg y combina nombre, sabor, envase, contenido y producto.

7. Ajustes de inventario

Archivo principal absoluto:

D:\xampp\htdocs\web3\ProyectoFuncional_7_17_2026_version12\9_11_2026_monitoreo_lacteo_sistema_ventas_funcional_v2\app\Services\AjusteInventarioServicio.php

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
---------------	------------------------	---------------	-------------	-----------------------------

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Listar ajustes	GET /ajustes-inventario . Controller: AjusteInventarioController . Modelo: AjusteInventario . Vista: ajustes-inventario/index.blade.php .	AjusteInventarioController::index()	SELECT ajustes_inventario con lotes, presentaciones, productos, productores y usuarios.	Menú → policy viewAny → consulta paginada → tabla de ajustes activos/anulados.
Formulario	GET /ajustes-inventario/create ; vista create.blade.php .	AjusteInventarioController::create()	SELECT lotes y relaciones.	Usuario autorizado abre formulario → sistema carga los lotes disponibles para seleccionar.
Crear ajuste	POST /ajustes-inventario . Request: GuardarAjusteInventarioRequest . Service: AjusteInventarioService .	store() → AjusteInventarioService::crear() ; auxiliares bloquear() , aplicar() , movimiento() .	TRANSACCIÓN: SELECT FOR UPDATE lote/presentación; INSERT ajustes_inventario; UPDATE lote y presentaciones; INSERT movimientos_inventario .	Envía cantidad física nueva y motivo → calcula diferencia contra saldo actual → actualiza lote y stock global → deja ajuste y movimiento auditado → COMMIT.
Editar ajuste activo	PUT /ajustes-inventario/{ajusteInventario} ; vista edit.blade.php ; Request ActualizarAjusteInventarioRequest .	update() → AjusteInventarioService::actualizar()	TRANSACCIÓN: bloqueos; UPDATE ajustes_inventario , lote y presentación; posible INSERT de movimiento correctivo.	Usuario cambia cantidad/motivo → calcula la corrección respecto a la diferencia original → aplica solo la variación adicional → conserva trazabilidad.
Anular ajuste	PATCH /ajustes-inventario/{ajusteInventario}/anular . Formulario en index/show. Request AnularAjusteInventarioRequest .	anular() → AjusteInventarioService::anular()	TRANSACCIÓN: UPDATE inverso de lote/presentación; UPDATE del ajuste a anulado; INSERT movimiento inverso.	Clic “Anular” → valida motivo → bloquea → calcula -\$ajuste->diferencia → revierte saldo sin borrar historial → marca quién/cuándo anuló → COMMIT.
Ver detalle	GET /ajustes-inventario/{ajusteInventario} ; vista show.blade.php .	AjusteInventarioController::show()	SELECT del ajuste y relaciones.	Clic “Ver” → policy view → carga responsables, lote y producto → muestra auditoría.

Diferencia entre anular y rollback: anular es una operación de negocio posterior que crea un movimiento inverso. Un rollback es técnico e inmediato: ocurre si falla una transacción antes del COMMIT.

8. Descartes de productos

Archivo principal absoluto:

D:\xampp\htdocs\web3\ProyectoFuncional_7_17_2026_version12\9_11_2026_monitoreo_lacteo_sistema_ventas_funcional_v2\app\Services\DescarteProductoService.php

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
Listar y filtrar	GET /descartes-productos . Controller DescarteProductoController . Service ConsultaDescartes . Vista descartes-productos/index.blade.php .	index() , filtros() , ConsultaDescartes::query()	SELECT descartes_productos y relaciones; SUM(perdida_total) .	Usuario filtra por proveedor, fechas, estado o motivo → consulta paginada → muestra pérdida consolidada.
Registrar pendiente	POST /descartes-productos . Vista _form.blade.php / create.blade.php . Request GuardarDescarteProductoRequest .	store() → DescarteProductoService::crear()	SELECT lote; INSERT descartes_productos . No descuenta stock todavía.	Usuario elige lote/cantidad/motivo → calcula costo y pérdida → guarda estado pendiente .
Actualizar pendiente	PUT/PATCH /descartes-productos/{descarteProducto} .	update() → DescarteProductoService::actualizar()	TRANSACCIÓN: SELECT FOR UPDATE presentación, lote y descarte; UPDATE descartes_productos .	Edita datos → bloquea registros → verifica que siga pendiente → recalcula costo/pérdida → COMMIT.
Eliminar pendiente	DELETE /descartes-productos/{descarteProducto} .	destroy() → DescarteProductoService::eliminar()	TRANSACCIÓN: bloqueos y DELETE físico en descartes_productos .	Clic eliminar → policy → bloquea → solo permite pendiente → borra → COMMIT; si ya fue procesado, excepción y rollback.
Procesar descarte	PATCH /descartes-productos/{descarteProducto}/procesar .	procesar() → DescarteProductoService::procesar()	TRANSACCIÓN: bloqueos; UPDATE/decrement lote y presentación; UPDATE descarte; INSERT movimientos_inventario .	Clic procesar → comprueba saldo de lote y general → descuenta ambos → marca responsable/fecha → registra kardex → COMMIT.
PDF individual/consolidado	GET .../{id}/pdf, GET .../reporte.pdf ; vistas pdf.blade.php, consolidado-pdf.blade.php .	pdf() , consolidadoPdf()	SELECT de descartes y relaciones.	Clic PDF → consulta → DomPDF renderiza → descarga; no cambia datos.

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo desde la GUI hasta BD
CSV	GET /descartes-productos/reporte.csv .	DescarteProductoController::csv()	SELECT ; salida streamDownload .	Clic CSV → aplica filtros → escribe filas al flujo de descarga; no cambia BD.

9. Proveedores y entradas de inventario

9.1 Proveedores

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo
Listar/buscar	GET /productores ; vistas productores/index.blade.php ; modelo Productor .	ProductorController::index()	SELECT productores , COUNT relacionado de ingresos_productores .	Usuario abre o busca → consulta activos o archivados → pagina resultados.
Crear	GET /productores/create , POST /productores ; vista create.blade.php , parcial _form.blade.php .	create() , store() , privado datos()	INSERT productores .	Formulario valida identidad/contacto/unidad productiva → crea proveedor → responde redirect o JSON.
Ver	GET /productores/{productor} ; vista show.blade.php .	show()	SELECT productores ; SELECT/COUNT últimas entradas/items .	Clic ver → carga proveedor e historial reciente de entradas.
Editar	GET .../edit , PUT/PATCH /productores/{productor} ; vista edit.blade.php .	edit() , update()	UPDATE productores .	Formulario → valida → actualiza → redirige o JSON.
Archivar	DELETE /productores/{productor} .	destroy()	UPDATE activo=false en productores ; no elimina historial.	Clic archivar → baja lógica → entradas permanecen relacionadas.
Restaurar	PATCH /productores/{productor}/restablecer ; solo Administrador.	restore()	posible restauración de deleted_at y UPDATE activo=true .	Administrador abre eliminados → restaura SoftDelete si existe → reactiva.

9.2 Entradas de inventario/acopio

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL y tabla	Flujo
Listar entradas	GET /ingresos-productores ; vista ingresos/index.blade.php ; modelos IngresoProductor , IngresoProductorItem .	ComercialController::ingresos()	SELECT ingresos_productores , productores , items .	Menú → consulta paginada → muestra entradas.
Preparar formulario	GET /ingresos-productores/create ; vista ingresos/create.blade.php ; JS comercial.js .	ComercialController::crearIngreso()	SELECT presentaciones/productos y productores activos.	Abre formulario → carga listas → JS permite agregar o quitar renglones.
Guardar entrada con lotes	POST /ingresos-productores ; Service ComercialService .	guardarIngreso() → ComercialService::ingreso()	TRANSACCIÓN: SELECT FOR UPDATE presentaciones / productores ; INSERT ingresos_productores ; múltiples INSERT ingresos_productores_items ; UPDATE presentaciones.stock .	Usuario envía cabecera e ítems → valida → bloquea presentaciones/proveedor → crea cabecera → crea cada lote con saldo inicial → suma stock general → COMMIT.
Ver detalle	GET /ingresos-productores/{ingreso} ; vista ingresos/show.blade.php .	ComercialController::detalle()	SELECT entrada , proveedor , operador , lotes , presentaciones y ventas .	Clic detalle → carga relaciones → muestra recepción, saldos y ventas por lote.
Previsualizar JSON	GET /ingresos-productores/{ingreso}/detalle-json ; JS comercial.js líneas de previsualización.	detalleJson() ; JS api() y manejador [data-view-ingreso] .	SELECT igual al detalle.	Clic previsualizar → fetch → JSON → modal sin navegación completa.
Exportar entrada	GET .../{ingreso}/pdf , GET .../{ingreso}/excel .	ReporteComercialController::pdfIngreso() , excelIngreso() , datos() , tablas() .	SELECT entrada , items , ventas y movimientos .	Clic exportar → consulta → DomPDF o Maatwebsite Excel → descarga.

10. Consignación histórica

Estado correcto para exponer

Consignación no es un módulo activo del esquema final. El sistema actual usa **proveedores + entradas de inventario + lotes**. No existen

ConsignacionController , modelo Consignacion ni vistas activas de consignación.

Elemento	Ubicación	Estado y función
Redirección /consignaciones	routes/web.php	Redirige a /ingresos-productores . No ejecuta SQL.
Redirección /mis-consignaciones	routes/web.php	Redirige a /ingresos-productores . No ejecuta SQL.
Redirección /consignaciones/{id}	routes/web.php	Redirige al detalle /ingresos-productores/{id} .

Elemento	Ubicación	Estado y función
Creación histórica	database/migrations/2026_09_05_000000_create_consignacion_y_ventas_tables.php	Creó consignaciones y consignacion_items en una versión anterior.
Fechas históricas	database/migrations/2026_09_08_000002_add_fechas_to_consignacion_items_table.php	Añadió fechas a ítems antiguos.
Refactor a acopio	database/migrations/2026_09_10_000001_refactor_acopio.php	Migra/renombra la estructura hacia productores , ingresos_productores e ingresos_productores_items .
Eliminaciones posteriores	2026_09_17_000001_remove_returns_and_integer_quantities.php y 2026_09_17_000003_remove_liquidaciones_completely.php	Retira devoluciones y liquidaciones históricas.

Frase recomendada: “El proyecto tuvo un diseño inicial de consignación, pero fue refactorizado. La implementación vigente trabaja con acopio: una entrada pertenece a un proveedor y contiene uno o más lotes. Las URLs antiguas se mantienen únicamente por compatibilidad.”

11. Productos y presentaciones

11.1 Productos

Funcionalidad	Archivos y ruta	Método exacto	SQL/tabla	Flujo
Listar/buscar	GET /productos ; ProductoController ; modelo Producto ; vista productos/index.blade.php .	ProductoController::index()	SELECT productos .	Menú → filtro activo/eliminado y nombre → paginación.
Crear	GET /productos/create , POST /productos ; vistas create.blade.php , form.blade.php .	create() , store() , validarProducto() , guardarImagenReferencial() .	INSERT productos ; archivo en storage/app/public/productos .	Formulario valida nombre, temperaturas, cuajo e imagen → guarda imagen → inserta producto.
Editar	GET .../edit , PUT /productos/{producto} .	edit() , update()	UPDATE productos ; posible reemplazo del archivo anterior.	Usuario modifica → valida → reemplaza imagen si envió una → actualiza.
Desactivar/restaurar	DELETE /productos/{producto} , PATCH .../restablecer .	destroy() , restore()	UPDATE activo=false/true ; no hay DELETE SQL.	Administrador desactiva sin perder historial; luego puede restaurar.

11.2 Presentaciones comerciales

Funcionalidad	Archivos y ruta	Método exacto	SQL/tabla	Flujo
Listar/buscar	GET /presentaciones ; PresentacionController ; vista presentaciones/index.blade.php ; servicio BusquedaPresentacion .	index() → BusquedaPresentacion::aplicar()	SELECT presentaciones , productos .	Filtro → búsqueda flexible → paginación.
Crear	POST /presentaciones ; vista create.blade.php , parcial form.blade.php .	store() , validar()	INSERT presentaciones con stock=0 ; imagen pública opcional.	Formulario define envase/sabor/contenido/unidad/precio → inserta.
Editar	PUT /presentaciones/{presentacion} .	update()	UPDATE presentaciones ; posible reemplazo de imagen.	Valida y actualiza descripción comercial; el stock no se toma del formulario.
Desactivar/restaurar	DELETE , PATCH .../restablecer .	destroy() , restore()	UPDATE activo ; al restaurar verifica que el producto padre esté activo.	Baja lógica protege relaciones; restauración exige producto activo.

Defensa: presentaciones.stock es un saldo global sincronizado con los saldos por lote. Los escritores críticos bloquean primero la presentación y luego el lote para reducir carreras y deadlocks.

12. Sensores IoT y API del ESP32

Controller absoluto:

D:\xampp\htdocs\web3\ProyectoFuncional_7_17_2026_version12\9_11_2026_monitoreo_lacteo_sistema_ventas_funcional_v2\app\Http\Controllers\Api\Esp32Controller.php

Contrato de rutas: routes/api.php ; todas llevan throttle:60,1 .

Funcionalidad	Ruta API / archivos	Método exacto	SQL y tabla	Flujo
Consultar estado	GET /api/estado ; también lo puede consumir dashboard.	Esp32Controller::estado()	SELECT actuadores , sensores , producciones .	ESP32 consulta → obtiene motor, ventilador, sensor y producción/objetivo en JSON.
Enviar temperatura	POST /api/temperatura .	Esp32Controller::temperatura()	UPDATE sensores.temperatura_actual ; SELECT/UPDATE producciones ; SELECT/INSERT alertas ; posibles UPDATE actuadores ; INSERT eventos . La serie corta también se guarda en caché.	Dispositivo envía temperatura → valida rango −50 a 150 → actualiza sensor → recalcula mínima/máxima/promedio/final → crea alerta si corresponde → puede apagar/finalizar → responde JSON.
Obtener temperaturas	GET /api/temperaturas , GET /dashboard/temperaturas , GET /obtenerDatosGrafico .	temperaturas() , obtenerDatosGrafico()	SELECT de producción para elegir clave; datos gráficos salen de caché.	Dashboard solicita periódicamente → obtiene hasta 20 puntos recientes.

Funcionalidad	Ruta API / archivos		Método exacto	SQL y tabla	Flujo
Ping del ESP32	POST /api/ping .	ping()		SELECT y UPDATE dispositivos (ultima_conexion , estado).	ESP32 envía MAC → valida registro → actualiza conexión → JSON de éxito o 404.
Datos del dashboard	GET /api/dashboard y GET /dashboard/datos .	dashboard()		SELECT en actuadores , sensores , producciones , alertas , dispositivos .	JS consulta → servidor compone estado completo → interfaz actualiza indicadores.
Recibir aviso de pasteurización	POST /api/alerta-pasteurizacion .	alertaPasteurizacion()		Actualmente SIN SQL ; valida y confirma JSON.	ESP32 envía mensaje/temperatura final → endpoint acepta y confirma; no crea fila en su implementación actual.
Finalizar automáticamente	POST /api/finalizar-produccion .	finalizarProduccionAutomatica()		SELECT/UPDATE producciones ; UPDATE actuadores ; INSERT eventos , alertas .	ESP32 indica final → termina producción, apaga equipos y crea alerta.
Aviso simple de pasteurización	POST /api/pasteurizacion .	pasteurizacion()		SIN SQL ; valida y responde.	Recibe temperatura y devuelve confirmación.

Aclaración sobre lecturas

Existe el modelo `Lectura` y la tabla `lecturas`, pero el método vigente `temperatura()` no inserta cada muestra en esa tabla: guarda la última temperatura en `sensores`, estadísticas en `producciones` y una serie corta en caché. Los reportes todavía pueden leer relaciones históricas de `lecturas`.

El código no identifica un modelo físico concreto del sensor (por ejemplo DS18B20); por tanto, no debe afirmarse como hecho durante la defensa.

13. Control de actuadores

Funcionalidad	Archivos, vista y ruta		Método exacto	SQL/tabla	Flujo
Ver panel	GET /control ; vista control/index.blade.php ; JS resources/js/control.js .		ControlController::index()	SELECT actuadores , sensores .	Usuario abre Control → carga motor, ventilador y sensor → muestra interruptores.
Motor manual	POST /control/motor/{estado} .		motor() → actualizarActuadorManual() → ActuadorService::actualizarEstadoManual() .	SELECT/UPDATE actuadores ; INSERT eventos .	Usuario cambia interruptor → ruta valida on/off → fija estado y modo Manual → audita.
Ventilador manual	POST /control/ventilador/{estado} .		ventilador() → mismo flujo del servicio.	SELECT/UPDATE actuadores ; INSERT eventos .	Igual al motor, usando tipo Ventilador.
Sensor	POST /control/sensor/{estado} .		ControlController::sensor()	SELECT/UPDATE sensores ; SELECT producción ; INSERT eventos .	Usuario activa/desactiva → cambia Activo/Inactivo → registra bitácora.
Cambiar modo	POST /control/{tipo}/modo .		cambiarModo() → ActuadorService::cambiarModo() .	SELECT/UPDATE actuadores ; INSERT eventos .	Switch de modo → convierte booleano a Manual/Automatico → actualiza y audita.
Encendido/apagado automático	Invocado desde producción e IoT.		encenderSiEstaEnAutomatico() , apagarAlFinalizar() , apagarAutomaticoSiEstaEncendido() .	SELECT/UPDATE actuadores ; INSERT eventos .	Inicio/final/temperatura invocan servicio → aplica regla del modo → registra acción.

Estas actualizaciones no están agrupadas en una transacción conjunta con los eventos; son operaciones secuenciales.

14. Dashboard, alertas y eventos

Funcionalidad	Archivos, vista y ruta		Método exacto	SQL/tabla	Flujo
Dashboard inicial	GET /dashboard ; DashboardController ; vista dashboard/index.blade.php .		DashboardController::index()	SELECT/COUNT productos , usuarios , dispositivos , producciones , lecturas , alertas , actuadores y sensores .	Usuario entra → servidor arma métricas iniciales → Blade renderiza tarjetas.
Refresco del dashboard	resources/js/dashboard.js ; GET /dashboard/datos , /dashboard/temperaturas .		JS actualizarDashboard() , actualizarGrafico() ; backend Esp32Controller::dashboard() , temperaturas() .	SELECT múltiples tablas; gráfico desde caché.	Temporizador JS → fetch → reemplaza estados, temperatura, alerta y gráfico sin recargar.
Listar alertas	GET /alertas ; AlertaController ; vista alertas/index.blade.php ; modelo Alerta .		AlertaController::index()	SELECT alertas , producciones , lecturas .	Menú → carga últimas alertas y relaciones → tabla.
Atender alerta	PATCH /alertas/{id}/atender .		AlertaController::atender()	SELECT y UPDATE alertas.atendida=1 .	Clic atender → busca alerta → guarda atendida → vuelve con mensaje.
Listar eventos	GET /eventos ; EventoController ; vista eventos/index.blade.php ; modelo Evento .		EventoController::index()	SELECT eventos , producciones , users ; filtros por tipo/fecha.	Usuario filtra → consulta paginada → muestra bitácora.
Registrar evento	Servicio app/Services/EventoService.php .		EventoService::registrar()	posible SELECT producciones ; INSERT eventos .	Cualquier operación IoT/producción/control llama al servicio → determina usuario → guarda tipo, descripción y fecha.

15. Reportes

15.1 Reportes de producción

Funcionalidad	Archivos/ruta	Método exacto	SQL/tabla	Flujo
Listado	GET /reportes ; ReporteController ; vista reportes/index.blade.php .	ReporteController::index()	SELECT producciones , users , productos .	Menú → lista producciones → acciones Ver/PDF.
Detalle	GET /reportes/{id} ; vista reportes/show.blade.php .	show() → ReporteProduccionService::calcularDatos() y formatearDuracion() .	SELECT producción, usuario, producto, dispositivo, alertas, eventos; posible COUNT lecturas .	Clic ver → carga datos → calcula duración y desviaciones → renderiza.
PDF	GET /reportes/{id}/pdf ; vista reportes/pdf.blade.php .	ReporteController::pdf()	Mismos SELECT ; DomPDF.	Clic PDF → consulta/cálculo → descarga.
Excel	GET /reportes/excel ; export app/Exports/ProduccionesExport.php .	ReporteController::excel() ; ProduccionesExport::collection() , headings() , styles() .	SELECT producciones y relaciones.	Clic Excel → construye colección → descarga XLSX.

15.2 Reportes comerciales

Funcionalidad	Archivos/ruta	Método exacto	SQL/tabla	Flujo
Ventas y ranking	GET /reportes-comerciales ; vista reportes/ventas.blade.php ; JS reporte-ventas.js .	ReporteComercialController::index() , tickets() ; ReporteVentasService::productosMasVendidos() .	SELECT tickets/ventas/lotos/productos/proveedores; ranking con JOIN , SUM , GROUP BY .	Usuario filtra periodo/producto/pro → consulta tickets → calcula top 15 → tabla/gráfico.
Ver comprobante JSON	GET /reportes- comerciales/ventas/{ticket} ; JS comercial.js .	ReporteComercialController::venta()	SELECT ticket, ventas, lotes, producto, proveedor, vendedor.	Clic ver → fetch → del comprobante.
Ranking CSV	GET /reportes- comerciales/productos.csv .	productosCsv() → productosMasVendidos() .	SELECT agregado ; descarga CSV.	Clic CSV → aplica periodo/proveedor → escribe ranking.
PDF comercial	GET /reportes- comerciales/pdf ; vista reportes/comercial- pdf.blade.php .	pdf() → datos()	SELECT entradas, ventas y movimientos.	Clic PDF → recopila conjuntos → genera apaisado.
Excel comercial	GET /reportes- comerciales/excel ; export ComercialExport .	excel() → datos() → tablas() → ComercialExport::sheets() .	SELECT ; no escribe BD.	Clic Excel → arma h Entradas, Ventas y A → descarga.
Reporte de ingresos	GET /reportes-ingresos ; vista reportes/ingresos.blade.php .	ingresosIndex() , privado ingresos()	SELECT ingresos_productores , items, presentaciones, productos y productores.	Usuario filtra → cons paginada → abre detalle/exportación.

La tabla reportes y el modelo Reporte existen para registros relacionados con producción, pero las pantallas actuales calculan/exportan directamente desde producciones y sus relaciones; los controladores mostrados no insertan filas nuevas en reportes .

16. Usuarios, autenticación y roles

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL/tabla	Flujo
Mostrar login	GET /login ; AuthController ; vista auth/login.blade.php .	AuthController::login()	Lectura de sesión; si ya está autenticado redirige. SELECT/exists users ; Auth::attempt	Usuario abre → formulario o redirección a producción.
Autenticar	POST /login , middleware throttle:5,1 .	AuthController::autenticar()	consulta credenciales y crea/regenera sesión.	Envía email/password → exige activo y rol interno → verifica hash → regenera sesión → producción.
Cerrar sesión	POST /logout .	AuthController::logout()	Invalida sesión; según driver puede afectar sessions .	Clic salir → logout → invalida sesión y CSRF → catálogo público.
Listar usuarios	GET /usuarios ; UserController ; vista usuarios/index.blade.php .	UserController::index()	SELECT users ; solo Administrador.	Busca activos/eliminados → pagina.
Crear usuario	POST /usuarios ; vista usuarios/create.blade.php .	store() , validarUsuario()	INSERT users ; contraseña con Hash::make .	Formulario → valida email único/rol/password → inserta.
Editar	PUT/PATCH /usuarios/{usuario} .	update()	UPDATE users .	Valida → cambia datos y hash solo si se envió contraseña.
Desactivar/restaurar	DELETE /usuarios/{usuario} , PATCH .../restablecer .	destroy() , restore()	UPDATE activo=false/true ; impide autoeliminación.	Administrador archiva/restaura sin perder relaciones.
Detalle individual no expuesto	El resource de usuarios usa - >except(['show']) ; no existe ruta GET ni vista de detalle individual.	No aplica UserController::show() .	SIN SQL .	La administración se realiza desde listado, creación y edición. Un GET directo a /usuarios/{id} es rechazado; PUT/PATCH/DELETE siguen disponibles para el CRUD.

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL/tabla	Flujo
Autorizar por rol	app/Http/Middleware/RolMiddleware.php ; alias en bootstrap/app.php .	RolMiddleware::handle()	SIN SQL adicional salvo usuario de sesión.	Ruta rol:Administrador,... compara rol y devuelve 403 si no coincide.
Autorizar comercio	app/Http/Middleware/OperadorComercial.php .	OperadorComercial::handle()	SIN SQL adicional.	Permite Administrador o Trabajador en POS, proveedores, inventario y reportes comerciales.
Policies	AjusteInventarioPolicy , DescarteProductoPolicy ; registro en AppServiceProvider .	viewAny , view , create , update , delete , process .	SIN SQL propio.	Controller llama authorize() → policy usa rol/estado → permite o 403.

17. Catálogo público

Funcionalidad	Archivos, vista y ruta	Método exacto	SQL/tabla	Flujo
Catálogo	GET /catalogo ; CatalogoController ; vistas catalogo/catalogo.blade.php , layout catalogo/layout.blade.php ; JS catalogo.js .	CatalogoController::index() ; JS aplicarFiltros() interno.	SELECT presentaciones y productos activos.	Visitante abre → servidor aplica filtros URL → renderiza tarjetas → JS permite filtros visuales adicionales.
Detalle	GET /catalogo/{presentacion} ; vista catalogo/detalle.blade.php .	CatalogoController::detalle()	SELECT presentación/producto activos.	Clic tarjeta → busca o 404 → muestra detalle.
Nosotros	GET /catalogo/nosotros ; vista nosotros.blade.php .	nosotros()	SIN SQL .	Clic → página informativa.
Contacto/proveedores	GET /catalogo/contacto ; vista contacto.blade.php .	contacto()	SELECT datos públicos seleccionados de productores activos.	Clic → consulta unidades con dirección → muestra contacto.

18. Transacciones, COMMIT, ROLLBACK y concurrencia

18.1 Dónde están las transacciones

Archivo y línea inicial	Método	Operaciones atómicas	COMMIT / ROLLBACK
app/Services/ComercialService.php:39	ingreso()	Cabecera de entrada + todos los lotes + stock de presentaciones.	Automáticos por DB::transaction(..., 3) .
app/Services/ComercialService.php:74	vender()	Ticket + ventas + descuento de todos los lotes + stock global + total.	Automáticos; hasta 3 intentos frente a deadlock.
app/Services/ComercialService.php:156	movimiento()	Saldo del lote + stock global + registro de movimiento.	Automáticos; 3 intentos.
app/Services/ComercialService.php:184	regularizar()	Datos faltantes del lote + movimiento auditado.	Automáticos; 3 intentos.
app/Services/AjusteInventarioService.php:16	crear()	Ajuste + lote + presentación + kardex.	Automáticos; 3 intentos.
app/Services/AjusteInventarioService.php:46	anular()	Movimiento inverso + estado del ajuste + kardex.	Automáticos; 3 intentos.
app/Services/AjusteInventarioService.php:71	actualizar()	Corrección del saldo + ajuste + kardex.	Automáticos; 3 intentos.
app/Services/DescarteProductoService.php:29	actualizar()	Validación/bloqueo y modificación del descarte pendiente.	Automáticos; 3 intentos.
app/Services/DescarteProductoService.php:53	eliminar()	Bloqueo y eliminación del descarte pendiente.	Automáticos; 3 intentos.
app/Services/DescarteProductoService.php:68	procesar()	Descuento lote/presentación + estado + kardex.	Automáticos; 3 intentos.
app/Http/Controllers/ProduccionController.php:59	iniciar()	Creación de la producción.	Automáticos; sin contador explícito de reintentos.
9_22_2026_monitoreo_lacteos.sql:11 y :1916	Volcado SQL	Importación completa del dump.	START TRANSACTION y COMMIT explícitos; no contiene ROLLBACK explícito.

18.2 Qué dispara el rollback

- Una ValidationException lanzada por reglas de negocio, por ejemplo stock insuficiente.
- Una excepción de base de datos: restricción única, clave foránea, deadlock no recuperado, etc.
- Cualquier otra excepción que salga del callback de DB::transaction() .
- Los cambios hechos dentro del callback se revierten juntos; los realizados antes o después no están cubiertos.

18.3 Bloqueos utilizados

lockForUpdate() genera un bloqueo equivalente a SELECT ... FOR UPDATE dentro de la transacción. Se usa en ventas, entradas, movimientos, ajustes y descartes. El orden predominante es:

1. Presentaciones ordenadas por ID.
2. Cabecera de ingreso cuando corresponde.
3. Lotes.
4. Registro de ajuste/descarte.

Esto reduce ventas dobles, saldos negativos y deadlocks por órdenes de bloqueo inconsistentes.

18.4 Operaciones importantes sin una transacción global

- `ProduccionController::finalizar()` actualiza producción, evento y actuadores secuencialmente.
- `Esp32Controller::temperatura()` puede actualizar sensor, producción, alertas, actuadores y eventos sin envolver todo en un `DB::transaction()` .
- `Esp32Controller::finalizarProduccionAutomatica()` también realiza varias escrituras secuenciales.
- `ActuadorService` actualiza actuador y luego inserta evento sin transacción conjunta.
- `DescarteProductoService::crear()` crea el pendiente sin transacción, porque aún no descuenta stock.

No significa que estas funciones no guarden datos; significa que sus múltiples escrituras no tienen rollback conjunto.

19. Mapa completo de tablas SQL

Tabla final	Modelo	Migración principal	Uso actual
users	User	0001_01_01_000000_create_users_table.php	Cuentas, rol, estado y relaciones de auditoría.
password_reset_tokens	Sin modelo	misma migración	Infraestructura Laravel para recuperación.
sessions	Sin modelo	misma migración	Sesiones cuando el driver configurado es database.
cache	Sin modelo	0001_01_01_000001_create_cache_table.php	Caché persistente si se usa driver database.
cache_locks	Sin modelo	misma migración	Bloqueos de caché.
jobs	Sin modelo	0001_01_01_000002_create_jobs_table.php	Cola de trabajos.
job_batches	Sin modelo	misma migración	Lotes de trabajos.
failed_jobs	Sin modelo	misma migración	Trabajos fallidos.
productos	Producto	2026_07_08_150320_create_productos_table.php + migraciones add_*	Producto lácteo, parámetros térmicos y cuajo.
dispositivos	Dispositivo	2026_07_08_150429_create_dispositivos_table.php	ESP32/MAC/última conexión.
sensores	Sensor	2026_07_08_150531_create_sensores_table.php	Estado y última temperatura.
actuadores	Actuador	2026_07_08_150720_create_actuadores_table.php	Motor/ventilador, estado y modo.
producciones	Produccion	2026_07_08_150744_create_producciones_table.php + ampliaciones	Lotes productivos y métricas térmicas.
lecturas	Lectura	2026_07_08_150851_create_lecturas_table.php	Lecturas históricas; el endpoint actual no inserta cada muestra aquí.
alertas	Alerta	2026_07_08_150909_create_alertas_table.php	Alertas y estado atendida.
reportes	Reporte	2026_07_08_150925_create_reportes_table.php	Registro relacionado con producción; exportación actual consulta producción directamente.
configuraciones	Configuracion	2026_07_08_150948_create_configuraciones_table.php	Configuración/umbrales del sistema.
eventos	Evento	2026_07_14_190918_create_eventos_table.php	Bitácora operativa.
presentaciones	Presentacion	2026_08_25_211829_create_presentaciones_table.php	Formato comercial, precio y stock global.
productores	Productor	2026_09_10_000001_refactor_acopio.php	Proveedores/unidades productivas.
ingresos_productores	IngresoProductor	misma migración	Cabecera de recepción/acopio.
ingresos_productores_items	IngresoProductorItem	misma migración + índices/enteros	Lotes, cantidades, precios y caducidad.
tickets_venta	TicketVenta	misma migración	Cabecera, UUID, hash, pago, cliente y total.
ventas	Venta	2026_09_05_000000_create_consignacion_y_ventas_tables.php + refactor	Líneas vendidas por lote.
movimientos_inventario	MovimientoInventario	2026_09_10_000001_refactor_acopio.php	Kardex/auditoría de cambios.
conciliaciones_stock_acopio	Sin modelo	misma migración	Conciliación técnica del refactor.
ajustes_inventario	AjusteInventario	2026_09_17_000004_create_ajustes_inventario_table.php	Ajustes y anulaciones auditadas.
descartes_productos	DescarteProducto	2026_09_17_000005_create_descartes_productos_table.php	Pérdidas pendientes/procesadas.

Tablas históricas que no deben presentarse como vigentes

`consignaciones`, `consignacion_items`, `liquidaciones` y `devoluciones` aparecen en migraciones de transición, pero fueron renombradas, migradas o eliminadas por migraciones posteriores.

20. Relaciones principales

User 1 — N Produccion / Evento / IngresoProductor / TicketVenta / Venta
User 1 — N AjusteInventario / DescarteProducto (según rol de auditoria)

Dispositivo 1 — N Sensor
Dispositivo 1 — N Actuador
Dispositivo 1 — N Produccion

Producto 1 — N Produccion
Producto 1 — N Presentacion

Productor 1 — N IngresoProductor

IngresoProductor 1 — N IngresoProductorItem
Presentacion 1 — N IngresoProductorItem
IngresoProductorItem 1 — N Venta
IngresoProductorItem 1 — N AjusteInventario
IngresoProductorItem 1 — N DescarteProducto
IngresoProductorItem 1 — N MovimientoInventario
TicketVenta 1 — N Venta
Produccion 1 — N Lectura / Alerta / Evento / Reporte
Sensor 1 — N Lectura
Lectura 1 — N Alerta

Modelos y métodos de relación

Modelo	Relaciones exactas
Actuador	dispositivo()
AjusteInventario	lote(), creador(), actualizador(), anulador()
Alerta	produccion(), lectura()
DescarteProducto	lote(), creador(), actualizador(), procesador()
Dispositivo	sensores(), actuadores(), producciones()
Evento	user(), produccion()
IngresoProductor	productor(), operador(), items()
IngresoProductorItem	ingresoProductor(), presentacion(), ventas(), ajustesInventario(), descartesProductos()
Lectura	produccion(), sensor(), alertas()
MovimientoInventario	lote(), responsable()
Presentacion	producto(), lotes()
Produccion	user(), producto(), dispositivo(), lecturas(), alertas(), eventos(), reportes()
Producto	producciones(), presentaciones()
Productor	ingresos()
Reporte	produccion()
Sensor	dispositivo(), lecturas()
TicketVenta	ventas(), vendedor()
User	producciones(), eventos(), ingresosRegistrados(), ventas(), ajustesInventarioCreados(), descartesProductosCreados()
Venta	lote(), ticket(), vendedor()

21. Rutas y archivos transversales

21.1 Archivos de entrada

Archivo	Función
public/index.php	Front controller HTTP de Laravel.
bootstrap/app.php	Registra rutas, middleware y alias.
routes/web.php	Catálogo, login y todas las pantallas autenticadas.
routes/api.php	Contrato HTTP para ESP32.
app/Http/Controllers/Controller.php	Clase base abstracta.
resources/views/layouts/app.blade.php	Layout interno con navbar/sidebar/contenido/footer.
resources/views/partials/sidebar.blade.php	Menú que conduce a cada módulo.
resources/js/app.js	Entrada JS; instala navegación parcial y confirmaciones.
resources/js/async-panel.js	Navegación y formularios asincrónicos internos.
vite.config.js	Entradas y compilación Vite.

La ruta raíz `GET /` se define como un closure en `routes/web.php` y redirige a la ruta nombrada `catalogo.index (/catalogo)`; no consulta la base de datos directamente.

21.2 Validación especializada

Archivo	Uso
GuardarAjusteInventarioRequest.php	Alta de ajuste.
ActualizarAjusteInventarioRequest.php	Modificación de ajuste activo.
AnularAjusteInventarioRequest.php	Motivo de anulación.
FiltroInventarioRequest.php	Filtros y detalle de inventario.
GuardarDescarteProductoRequest.php	Alta/edición de descarte.

21.3 Frontend JavaScript

Archivo	Funciones relevantes
resources/js/comercial.js	<code>initializeCommercial()</code> , <code>api()</code> , <code>load()</code> , <code>pay()</code> , <code>allocations()</code> , <code>setQty()</code> , <code>render()</code> , <code>buildPrintableTicket()</code> .
resources/js/supplier-cart.js	Selección/cálculo de proveedor y stock en pruebas/helpers.
resources/js/dashboard.js	<code>actualizarDashboard()</code> , <code>actualizarGrafico()</code> , alarma visual/sonora.
resources/js/control.js	<code>initializeControl()</code> y envío de cambios del panel.
resources/js/produccion.js	Formulario y recomendación de cuajo.

Archivo

`resources/js/rennet-recommendation.js` `requiredRennet()` , `rennetReferences()` .

`resources/js/catalogo.js`

`resources/js/reporte-ventas.js`

`resources/js/async-panel.js`

`resources/js/responsive-tables.js`

Funciones relevantes

Filtros del catálogo ya renderizado.

Gráfico/ranking de productos vendidos.

`navigatePanel()` , `submitAsyncAction()` , actualización parcial de `<main>` .

Adaptación de tablas a pantallas pequeñas.

21.4 Rutas de compatibilidad

URL antigua	Destino vigente
<code>/consignaciones</code>	<code>/ingresos-productores</code>
<code>/mis-consignaciones</code>	<code>/ingresos-productores</code>
<code>/consignaciones/{id}</code>	<code>/ingresos-productores/{id}</code>
<code>/reportes-inventario</code>	<code>/reportes-comerciales</code>

21.5 Protección de acceso

- Catálogo, nosotros, contacto y login son públicos.
- Dashboard, producción, control, alertas, eventos y reportes de producción usan `auth` .
- Productos/presentaciones usan `auth` más `rol:Administrador,Trabajador` ; desactivar/restaurar se limita a Administrador.
- Usuarios se limita a Administrador.
- POS, proveedores, entradas, inventario, ajustes, descartes y reportes comerciales usan `OperadorComercial` .
- La API del ESP32 usa limitación de 60 solicitudes por minuto; el código mostrado no aplica autenticación por token.

22. Resumen para la defensa

22.1 Explicación corta de arquitectura

“El sistema sigue una arquitectura MVC de Laravel. La vista Blade y JavaScript generan la petición; `routes/web.php` o `routes/api.php` selecciona el controlador; el controlador valida y delega las reglas complejas a servicios; los modelos Eloquent representan las tablas MySQL. Las operaciones comerciales críticas usan transacciones y bloqueos de fila.”

22.2 Explicación del carrito

“El carrito vive temporalmente en un `Map` de JavaScript y no afecta la base de datos mientras el usuario agrega productos. Al cobrar, se envía todo el carrito con una clave UUID. El servidor abre una transacción, bloquea existencias, aplica FEFO por fecha de caducidad, crea ticket y ventas, descuenta los lotes y actualiza el stock general. Si un artículo falla, se hace rollback de todo el carrito.”

22.3 Explicación de inventario

“La fuente detallada es `ingresos_productores_items` , donde cada fila es un lote. `presentaciones.stock` mantiene el consolidado global. Los servicios modifican ambos saldos dentro de la misma transacción y escriben `movimientos_inventario` como kardex.”

22.4 Explicación de consignación

“Consignación pertenece al diseño histórico. Fue reemplazada por acopio: proveedores, entradas y lotes. Las rutas antiguas solo redirigen al módulo vigente y las migraciones explican la evolución del esquema.”

22.5 Explicación de IoT

“El ESP32 consume una API JSON. Envía ping y temperatura y consulta el estado. Laravel actualiza el sensor y estadísticas de producción, controla alertas y puede apagar actuadores o finalizar el proceso. El dashboard consulta esos endpoints y actualiza la interfaz de forma periódica.”

22.6 Explicación de COMMIT y ROLLBACK

“En Laravel no se escriben necesariamente las palabras COMMIT y ROLLBACK en cada método. `DB::transaction()` las gestiona: si el callback termina, hace COMMIT; si lanza una excepción, hace ROLLBACK. El segundo parámetro `3` permite reintentar hasta tres veces ciertos conflictos como deadlocks.”

22.7 Puntos que conviene no afirmar incorrectamente

- No afirmar que consignación sigue activa.
- No afirmar que el carrito del navegador decide el lote definitivo; el servidor decide FEFO.
- No afirmar que cada lectura térmica se inserta actualmente en `lecturas` ; la implementación vigente usa sensor, producción y caché.
- No afirmar un modelo físico específico de sensor si no se aporta el firmware/documentación externa.
- No confundir anulación de un ajuste con rollback técnico.
- No afirmar que todas las operaciones IoT están en transacciones; varias son escrituras secuenciales.

Fuente de verdad revisada

- `GUIA_SISTEMA.md` como orientación funcional.
- `routes/web.php` y `routes/api.php` como contrato HTTP.
- `Controllers` y `Services` bajo `app/` como lógica efectiva.
- `Models` y migraciones como estructura de datos.
- `Vistas` y `JavaScript` bajo `resources/` como inicio real de los flujos de usuario.
- `9_22_2026_monitoreo_lacteos.sql` como volcado SQL disponible en la raíz.

En caso de discrepancia entre una descripción histórica y el código activo, este documento prioriza el código activo y marca expresamente lo histórico.